

GROUP MEETING • PAPER READING

从 prompt 到 context 到 *harness*

Agent Harness Engineering: A Survey

Li et al., 2026 · TMLR under review · 覆盖 170+ 开源项目

外加 infra 视角: FlashML / Mooncake / TrEnv-X 等系统侧工作

一篇综述,一个核心命题

agent 的可靠性瓶颈,正在从模型本身,转移到包裹模型的 harness。

- | | | |
|----|------------------------------|------------------------------|
| 01 | 命题:harness 是当前的瓶颈 | §1 三个实验 + binding constraint |
| 02 | 框架:harness 怎么拆成七层 | §2 三阶段演化 + ETCLOVG |
| 03 | 细看 Context 与 Verification 两层 | §5 / §8 |
| 04 | infra 视角:harness 的计算开销 | FlashML / 异构 / 算子加速 |
| 05 | 模型变强后,harness 会过时吗 | §12.5 + 判断 |

PART · 01

决定 agent 好不好用的， 是模型，还是 *harness*？

过去两年的研究几乎都在问“模型能做什么”。

这篇综述给出一个不同的答案。

模型不动,只改 harness,提升超过一次模型迭代

三个 2026 年的实验:固定模型,只动外围基础设施

Bölük 2026

10×

Coding benchmark

只改 edit-tool 的格式和外围 harness,模型一行没动。跨 15 个模型,最高 10× 提升。

Trivedy 2026 · LangChain

+13.7pp

Terminal-Bench 2.0

固定 GPT-5.2-Codex,只改 system prompt、middleware、self-verification。52.8% → 66.5%。

Lee et al. 2026

76.4%

Meta-Harness 自动搜索

让程序自动搜索 harness 配置,在 Terminal-Bench-2 上超过所有手工方案。

一次模型大版本升级,同 benchmark 通常带来 2-4 pp;只改 harness 的收益,反而更大。

binding constraint:此刻真正卡住你的那个约束

和系统里的 *memory-bound* / *compute-bound* 是同一个意思

论文的命题

对在能力相当的前沿模型上评测的长 horizon 任务,benchmark 的方差可能更多来自 execution harness,而不亚于来自模型本身。

原文:"benchmark variance may be driven as much by the execution harness as by the model itself."

系统里瓶颈会转移:程序优化到一定程度,会从 compute-bound 变成 memory-bound。harness 也一样,在这一代模型上,边际收益最大的约束恰好落在 harness; 模型再变强,它可能又移到别处。

业界在实践,学术界在研究组件

practitioner 早知道 *harness* 重要,缺的是说清“为什么”的学术词汇

PRACTITIONER 怎么说

OpenAI

把 harness engineering 定义为一门学科:围绕 agent 设计环境、约束、文档、反馈回路。5 人团队 5 个月写了约百万行内部代码。

Anthropic

架构要简单可检查;工具为 agent 设计而非照搬 human API;context 渐进披露;长任务要可恢复执行。

Martin Fowler

harness = “AI agent 的控制论调节器”:前馈引导 + 反馈传感,构成围绕 LLM 的控制回路。

RESEARCH 在做什么

学术界在越来越精确地研究 **memory**、**tool use**、**planning**、**safety** 这些组件。

但很少有人系统研究把这些组件整合成可靠运行的那个系统，也就是 harness 本身。

这篇综述的价值在于第一次把 harness engineering 命名成独立的研究对象。

PART · 02

harness 怎么拆开 理解

先看它从哪一步步演化来,再看作者提出的七层划分。

prompt → context → harness

三个阶段,是“边际工程努力流向哪里”的转移

2022 — 2024 Prompt 优化单次调用的输入文本 few-shot · CoT · 模板	2025 Context 优化每一步该让模型看什么 注入 · 检索 · 压缩	2026 — Harness 优化包裹模型的整个 wrapper 七层 ETCLOVG
--	--	---

三者是包含关系,不是替代 —— harness $\supset$ context $\supset$ prompt。prompt engineering 没有过时,它仍是 harness 实践的一部分,只是不再是边际收益最大的地方。

ETCLOVG:把 harness 拆成七层

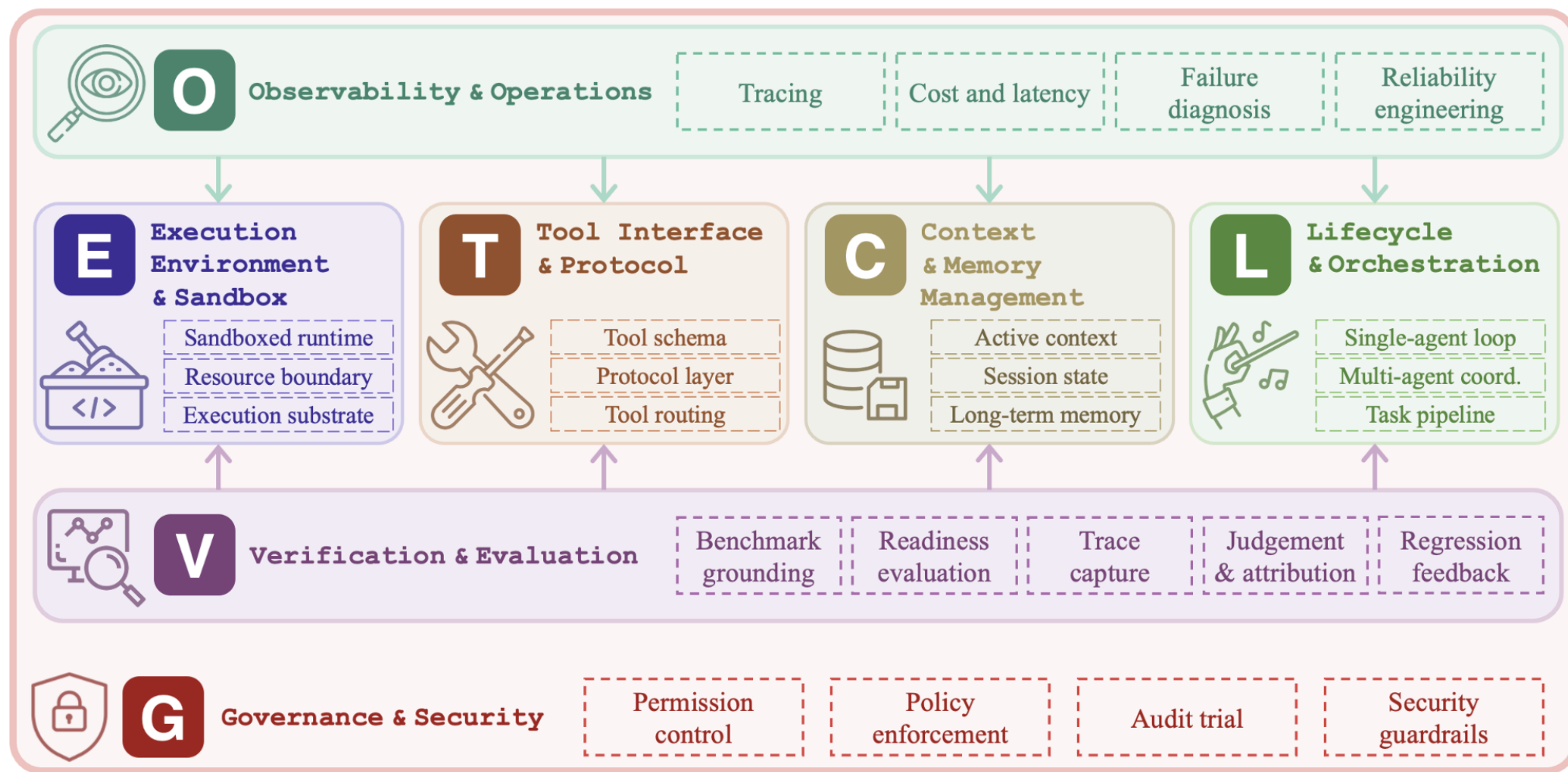
前四层是结构核心,后三层是控制平面



论文的两个改动

把 **Observability** 和 **Governance** 从“lifecycle hook 的副作用”提升为一级层 —— 它们已经有独立的工具生态和团队。不过这套分类是 descriptive 的:它描述业界怎么切,没回答“为什么必须七层”;论文 §13 也承认,把它变成 normative 框架是 future work。

七层之间的结构关系



A Timeline of Harness Systems

Key milestones across Execution, Tools, Context, Orchestration, Observability, Evaluation, and Governance



PART · 03

细看两层:*Context* 与 *Verification*

Context: 模型每一步该看到什么 —— 记忆怎么管、上下文怎么组织。

Verification: 怎么判断 *agent* 做得对不对 —— 评估、失败归因、回归。

更大的窗口,不解决记忆问题

三个机制层面的理由:context 是稀缺资源,不是免费的

机制 01

注意力是平方成本

$O(n^2)$

context 翻倍,成本翻 4 倍。FlashAttention 降的是常数,不是结构。

机制 02 · Liu 2024

Lost in the middle

-30%

相关信息放在中间,准确率比放头尾掉 30% 以上。所有模型都有。

机制 03 · Hong 2025

Context rot

200K→50K

18 个前沿模型全部退化。标称 200K 的窗口,可能 50K 就开始掉。

瓶颈不是窗口多大,而是在有限的注意力预算里放什么、放哪、何时移除。这就是 context engineering。

短期:管好当前这一个 context window

三类技术,直接决定一次调用的成本和质量

KV-cache 友好布局

缓存命中比未命中便宜约 10×(\$0.30 vs \$3.00/MTok)。规则:前缀稳定、只追加不修改、序列化确定。Manus 禁用工具时不删工具列表,而是 decoding 时 mask logits,避免改前缀。

Progressive disclosure

不一次性塞进所有可能相关的信息。只保留文件路径、查询、URL 这类轻量标识符,需要时再加载。Claude Code 的 CLAUDE.md + grep/glob 就是这样。

System prompt 校准

找“正确高度”:太具体则脆弱,太模糊则没指导。用最小 prompt 配最强模型起步,经验性地发现失败模式再针对性补,而不是预先枚举所有 edge case。

中期:跨回合、跨运行保住状态

几百个 *token* 的工作状态,就能跨过一次 *context* 清空,不丢进展

结构化笔记

agent 维护一个 NOTES.md,每次运行开始读、清空前写。Claude 玩宝可梦几千步,没人教它怎么记,它自己发展出地图、战斗统计、目标列表,每次压缩后读笔记继续。

File-based planning

把整个规划结构写到磁盘:任务状态、规格、中间结果、依赖图。下次按需加载,不立即相关的内容完全绕过 context window。

Cross-run injection

抓取上一次运行的关键输出,拼到下一次开头。claude-mem 这类工具不需要向量库,就能提供相当好的跨运行连续性。

核心思路:把工作记忆外化

不指望对话历史扛住状态,而是写到外部、需要时读回。轻量、无需数据库;代价是只能向前推送、不能任意检索,历史一多就要上长期记忆。

长期:带索引、可检索的记忆系统

五个系统,各攻一个不同的问题

MemGPT

把 context 当 RAM、外部存储当 disk,给模型 function call 显式分页。后续记忆系统的范式起点。

Memo

部署最广。向量库 + 图库 + KV 三后端混合,LLM 抽取分流。比 OpenAI 原生记忆准 26%、省 90% token;AWS 选为 Agent SDK 独家记忆。

A-MEM

借鉴 Zettelkasten:新记忆进来时回头更新旧记忆的连接。解决“一条记忆的重要性,往往事后才显现”。

Honcho

建“用户模型”而非“用户事实”。后台异步 dreaming 推断偏好;多 agent 共享一个用户而不互相污染。

cq

agent 之间的共享记忆。出错时自动查集体知识库,防止整个 fleet 重复踩同一个坑。

学术分类:write-read-manage 循环(Zhang 2025);Pattern A/B/C 对应短/中/长三层(Du 2026)。

100+ 轮之后:压缩、隔离,与没解决的 drift

技术能缓解长任务,但有一个问题留到了最后

Compaction(压缩)

接近窗口上限时压缩重启。先尽量保全(maximize recall),再迭代去冗余;反过来做会丢掉找不回的信息。

Sub-agent 隔离

把子任务交给独立 agent,各自 fresh context,只回传 1-2K token 的浓缩结论。详细探索隔离在子 agent 内,不污染主流程。

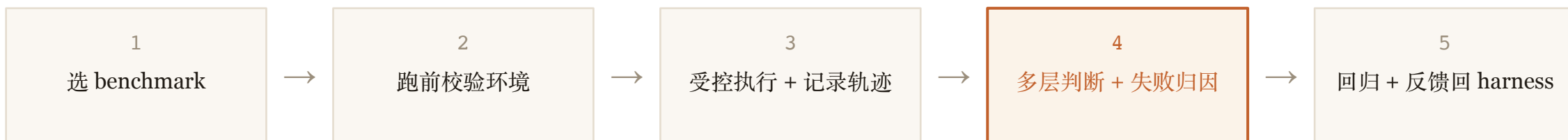
Context drift —— 未解

跑过 100 轮后,agent 会重复已做的工作、和早期决策无意识矛盾、跑偏目标。根因不是窗口满,而是每次压缩的细微不准确累积,而当前架构无法检测这种发散。

Context 单靠自己解决不了长 horizon 可靠性,这正是后面 Verification、Observability 存在的理由。

评估是一个闭环,不是打个分就完事

五个阶段,核心矛盾在第四步:失败到底出在哪一层



现在大多数 agent benchmark 只给一个 **pass / fail**。这个数字对调试系统几乎没用——失败到底是 planning 错了、tool 错了、context 丢了,还是 sandbox 限制了?不归因到 ETCLOVG 的某一层,就没法系统改进。论文还强调:evaluator 本身(比如 LLM-as-judge)也要被当作“被测组件”,它有 position bias、verbosity bias,不是天然可信的 oracle。

一个能看见的研究空白

LANGCHAIN 2026 调查

89%

团队用 observability

52%

才做 offline eval

大家都在看 *agent* 做了什么,只有一半的人系统判断它做得对不对。

可做的方向

未来的 agent benchmark 不该只给 final score,而要给分层的 failure attribution —— 把每次失败归到 ETCLOVG 的某一层。

更进一步:能不能为 harness 设计 unit-test 式的评估,而不是只能端到端测(端到端包含太多 confounding)?

PART · 04

infra 视角:*harness* 的计算账

综述主要从软件工程角度讲 *harness*。

但把 *agent* 端到端的开销摊开看,模型推理之外的部分正在变成大头。

一个 agent 跑起来,计算花在哪

模型推理(prefill/decode)只是其中一环

检索 retrieval

向量最近邻(KNN/ANN)。RAG、长期记忆、工具选择都要它。带宽密集,且在用户等待的关键路径上。

重排 / 聚类 / 去重

rerank 打分、KMeans / HDBSCAN 归并相似结果。原是离线分析,现在进了实时循环。算力密集。

验证 verification

跑测试、打分、算 PCA/SVD/最近邻判断结果对不对。在“生成-验证-再生成”里反复跑。

上下文组装 / KV 存取

拼 context、序列化;KV-cache 在 HBM/DRAM/SSD 之间取放、压缩。带宽 + 存储密集。

环境 / 工具启动

sandbox 起容器、装依赖、初始化。TrEnv-X:这类开销可占端到端成本的 70%。

调度 / 状态管理

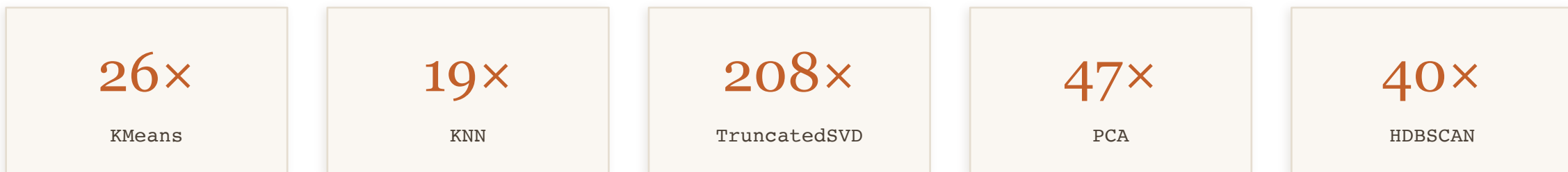
决定哪步在哪跑、维护和传递 agent 状态。长 horizon 里持续发生。

Agent 越复杂,模型推理之外的这些开销加起来,可能比推理本身还重。

瓶颈从 transformer 扩展到模型周围的计算

FlashML(UC Berkeley / MIT 等):classical ML operator 正进入 agent 关键路径

过去 MLSys 是 model-centric 的:大家优化 transformer 核 —— FlashAttention、FlashDecoding、KV-cache。但 agent 把模型变成了一个**控制器**,瓶颈扩展到模型周围的整个计算基底:检索、聚类、PCA、SVD、验证。这些 classical operator 的实现还停留在 pre-Hopper、pre-agent 时代。



把这些算子在现代 GPU 上重写,相比 cuML 的加速。flash-KNN 能打满 H200 的 85% HBM 带宽。

“faster intelligence assembly, not just faster LLM inference.” —— 这正是算子加速能直接做的。

按资源把开销分类,机会就清楚了

不同开销吃不同资源,对应不同的系统优化

算力密集

聚类、PCA/SVD、重排打分

在 GPU 上重写算子 —— FlashML 那条线

带宽密集

向量检索、KV-cache 存取

让数据流动更快、放对存储层 —— Mooncake 式调度

容量密集

大向量库、历史 KV 卸载、长期记忆

大容量 + 中等带宽 + 不需顶级算力 —— 带 HBM 的 CPU 这类异构硬件的甜区

环境 / 启动

sandbox、工具环境

可复用、可共享的执行环境 —— TrEnv-X

做 GPU 算子加速,正好对着 harness 里算力密集的那一类:检索、聚类、验证。

PART · 05

模型变强后， *harness* 会过时吗

如果紧约束会转移,那 *harness engineering* 本身会不会被模型吃掉?

会过时的是“补能力”的部分,不是“管边界”的部分

论文 §12.5 给了一个可操作的判断框架

ANTHROPIC:OPUS 4.5 → 4.6

模型变强后,他们**移除了** sprint 结构和 context reset,成本从 **\$200 降到 \$125**,质量不变。

harness-as-assumption:每个 *harness* 组件,都编码了一个“模型自己做不到什么”的假设。模型变强,假设就过期。

两种 **harness**,命运不同

补足模型能力的 —— 显式 ReAct 模板、繁琐 self-check、为弱模型设的 context reset:会被模型吃掉。

管理执行边界的 —— sandbox、permission、audit、**verification**:跟模型多强正交,只会随 agent 自主性上升而更重要。

三个 **harness-only** 实验都在前沿模型上做的 —— 即使最强的模型,**harness** 的边际收益依然很大。

Four takeaways

01 综述的贡献是“命名”,不是分类

第一次把 harness engineering 立成独立研究对象,把散在工程博客里的实践知识翻成研究词汇。

02 瓶颈会转移,但 harness 不会消失

会过期的是补能力的部分;管边界的部分(governance / verification)随自主性上升而更重要。

03 Context 有直接可用的工程

KV-cache、progressive disclosure、记忆分层 —— 对所有用 LLM 的人都有用,不只 agent。

04 infra 的机会:模型之外的计算

检索、聚类、验证、KV 存取进了关键路径。做 GPU 算子加速,正对着算力密集的那一类。

Q & A

欢迎讨论

Agent Harness Engineering: A Survey · Li et al. 2026 · 外加 FlashML / Mooncake / TrEnv-X 等系统侧工作